📌 STRING FUNCTIONS IN MYSQL

1. Concatenation Functions

Used to join multiple strings into one.

- **`CONCAT(str1, str2, ...)`**: Joins two or more strings.
  - *Example:* `SELECT CONCAT('MySQL', ' ', 'Functions');`
  - *Output:* `'MySQL Functions'`
- **`CONCAT_WS(separator, str1, str2, ...)`**: Concatenates strings with a specified separator.
  - *Example:* `SELECT CONCAT_WS('-', '2026', '01', '02');`
  - *Output:* `'2026-01-02'`

2. Substring & Extraction Functions

Used to extract specific parts of a string.

- **`SUBSTRING(str, start, length)`**: Extracts a portion of a string starting from a specific position.
  - *Example:* `SELECT SUBSTRING('Database', 1, 4);`
  - *Output:* `'Data'`
- **`LEFT(str, len)`**: Returns the leftmost characters.
  - *Example:* `SELECT LEFT('MySQL', 2);`
  - *Output:* `'My'`
- **`RIGHT(str, len)`**: Returns the rightmost characters.
  - *Example:* `SELECT RIGHT('MySQL', 3);`
  - *Output:* `'SQL'`

3. Length & Search Functions

Used to find the size or position of text.

- **`LENGTH(str)`**: Returns the length of a string in **bytes**.
  - *Example:* `SELECT LENGTH('text');`
  - *Output:* `4`
- **`CHAR_LENGTH(str)`**: Returns the length of a string in **characters** (useful for multi-byte characters).
  - *Example:* `SELECT CHAR_LENGTH('©');`
  - *Output:* `1` (while `LENGTH` might return 2)
- **`LOCATE(substr, str)`**: Returns the position of the first occurrence of a substring.
  - *Example:* `SELECT LOCATE('a', 'banana');`
  - *Output:* `2`

4. Transformation & Formatting

Used to change the appearance or content of a string.

- **`UPPER(str)`** / **`LOWER(str)`**: Converts text to uppercase or lowercase.
  - *Example:* `SELECT UPPER('mysql');`
  - *Output:* `'MYSQL'`
- **`REPLACE(str, from_str, to_str)`**: Replaces all occurrences of a substring with another.
  - *Example:* `SELECT REPLACE('Apple Inc.', 'Inc.', 'Corp.');`
  - *Output:* `'Apple Corp.'`
- **`REVERSE(str)`**: Reverses the characters in a string.
  - *Example:* `SELECT REVERSE('MySQL');`
  - *Output:* `'LQSyM'`

5. Trimming & Padding

Used to remove or add characters around a string.

- **`TRIM(str)`** : Removes leading and trailing spaces.
  - *Example:* `SELECT TRIM(' hello ');`
  - *Output:* `'hello'`

Example Query	Input `str`	Output `trimmed_string`	Explanation
`SELECT TRIM(' Hello World ');`	`' Hello World '`	`'Hello World'`	Removes leading and trailing spaces by default.
`SELECT TRIM(LEADING 'x' FROM 'xxxDataxxx');`	`'xxxDataxxx'`	`'Dataxxx'`	Removes only the leading 'x' characters.
`SELECT TRIM(TRAILING 'x' FROM 'xxxDataxxx');`	`'xxxDataxxx'`	`'xxxData'`	Removes only the trailing 'x' characters.
`SELECT TRIM(BOTH 'x' FROM 'xxxDataxxx');`	`'xxxDataxxx'`	`'Data'`	Removes 'x' characters from both ends.
`SELECT TRIM(LEADING '0' FROM '007 Code Name');`	`'007 Code Name'`	`'7 Code Name'`	Strips leading zeros.
`SELECT TRIM(' \\t\\n Data\\n\\r ');`	`' \\t\\n Data\\n\\r '`	`' Data\\n\\r '`	Note: `TRIM()` by default only removes space (ASCII 32) characters, not other whitespace like tabs (`\t`) or newlines (`\n`).

  -

- **`LPAD(str, len, padstr)`**: Pads the left side of a string to a certain length.
  - *Example:* `SELECT LPAD('123', 5, '0');`
  - *Output:* `'00123'`

1. Pattern Matching & Regex
These functions use regular expressions (Regex) to search or modify text.

- **`REGEXP_LIKE(str, pat)`**: Returns 1 if a string matches a regular expression pattern.
  - *Example:* `SELECT REGEXP_LIKE('MySQL 8.0', '[0-9]');`
  - *Output:* `1`
- **`REGEXP_REPLACE(str, pat, repl)`**: Replaces occurrences matching a pattern.
  - *Example:* `SELECT REGEXP_REPLACE('Apples 10, Oranges 20', '[0-9]+', 'XX');`
  - *Output:* `'Apples XX, Oranges XX'`
- **`SUBSTRING_INDEX(str, delim, count)`**: Returns a substring before a specific number of occurrences of a delimiter.
  - *Example:* `SELECT SUBSTRING_INDEX('www.mysql.com', '.', 2);`
  - *Output:* `'www.mysql'`

2. Set & List Functions
Useful for managing comma-separated values or choosing from lists.

- **`FIND_IN_SET(str, strlist)`**: Returns the index position of a string within a comma-separated list.
  - *Example:* `SELECT FIND_IN_SET('b', 'a,b,c,d');`
  - *Output:* `2`
- **`ELT(N, str1, str2, ...)`**: Returns the Nthcap N raised to the t h power

  Nth string from a list of arguments.

  - *Example:* `SELECT ELT(3, 'Apple', 'Banana', 'Cherry');`
  - *Output:* `'Cherry'`
- **`FIELD(str, str1, str2, ...)`**: Returns the index of `str` in the provided list (opposite of `ELT`).
  - *Example:* `SELECT FIELD('Banana', 'Apple', 'Banana', 'Cherry');`
  - *Output:* `2`

3. Comparison Functions
Used to evaluate how strings relate or sound phonetically.

- **`STRCMP(str1, str2)`**: Returns `0` if strings are equal, `-1` if the first is smaller, and `1` if the first is larger.

- *Example:* `SELECT STRCMP('text', 'text2');`
- *Output:* `-1`

4. Encoding & Numeric Conversion

- **`ASCII(str)`**: Returns the numeric ASCII value of the leftmost character.
  - *Example:* `SELECT ASCII('A');`
  - *Output:* `65`

Search & Positioning Synonyms

MySQL offers multiple ways to find the location of a substring.

- **`INSTR(str, substr)`**: Returns the position of the first occurrence of a substring (reverse argument order of `LOCATE`).
- *Example:* `SELECT INSTR('Hello World', 'Wor');`
- *Output:* `7`
- **`POSITION(substr IN str)`**: A standard SQL synonym for `LOCATE()`.
- *Example:* `SELECT POSITION('bar' IN 'foobarbar');`
- *Output:* `4`

3. Substring Extraction Synonyms

- **`MID(str, pos, len)`**: A synonym for `SUBSTRING(str, pos, len)`.
- *Example:* `SELECT MID('Hello World', 7, 5);`
- *Output:* `'World'`

Formatting & Conversion

- **`FORMAT(X, D [, locale])`**: Formats a number `X` to a string like `#,###,###.##`, rounded to `D` decimal places.
- *Example:* `SELECT FORMAT(1234567.89, 2);`
- *Output:* `'1,234,567.89'`

Padding & Spacing

- **`RPAD(str, len, padstr)`**: Right-pads a string to a certain length.
  - *Example:* `SELECT RPAD('hi', 5, '?');`
  - *Output:* `'hi???'`